

Song-Lyrics Genre Classification with Classical Models, Recurrent Networks, BERT, and RoBERTa

Md Nazim Hossain

Department of CSE

Brac University

md.nazim.hossain@g.bracu.ac.bd

Abstract

This is a study of multi-class genre classification from song-lyrics across 6 different labels: Dance, Heavy Metal, Hip Hop, Indie, Pop, and Rock. A public English-labeled corpus from huggingface was cleaned, dropped duplicates, and separated into stratified training (80%), validation (10%), and test (10%) partitions. Both TF-IDF and Word2Vec representations were implemented for comparative analysis with classic ML models like logistic regression, Naive Bayes, and random forest classifiers with three different configurations. Six recurrent architectures were evaluated also with three different configurations. Finally, three transformer models were trained, named in the project as unweighted BERT, weighted BERT, and weighted RoBERTa. A three-model probability ensemble technique was introduced to tackle the severe class imbalance for the highest performance gain. The dataset was heavily pre-processed for ML models, but minimally normalized for sequence and transformer models. On the test set, the ensemble of three models performed best with a macro F1-score of 0.56 at 0.66 accuracy. Weighted RoBERTa turned out to be the strongest single transformer at a macro F1-score of 0.55 and 0.64 accuracy. Among the classical models, Logistic regression gave the strongest macro F1 (0.41), while bidirectional GRU was the strongest recurrent model (0.37). Contextual pretraining proved to be beneficial for better performance, but errors caused by severe class imbalance in the dataset, overlapping language among different genres, truncation, and sequence handling were prominent.

1 Introduction

Music genre is mainly expressed by acoustic properties, but vocabulary, sentence structure, narrative, and lyrical structure also carry a significant impact. Detecting genre using lyrics alone is difficult but a meaningful multi-class natural language processing

task. It can help catalog organization, recommendation, search, and music analysis on a greater scale when audio is unavailable or used as complementary evidence. The difficulty of the task is uplifted by adjacent genres sharing themes and expressions, genre boundaries being socially defined, and the probability of a single song belonging to more than one category or overlapping boundaries.

It was established by earlier research that word use, rhyme, stylistic features, and higher-level lyrical structure carry genre information (Mayer et al., 2008; Fell and Sporleder, 2014). More recent work has proven that recurrent and attention-based models can learn useful information from intact lyrics (Tsaptsinos, 2017). Our work builds an experimental setup focused on this analysis. TF-IDF vectors, Word2Vec embeddings, recurrent neural networks, pretrained transformers, and an ensemble technique show the effects of representation, architecture, preprocessing, and class weighting on the same split.

Three significant research questions are addressed in this study. One, How do the performance of classical, recurrent, and transformer-based models compare under a common test protocol? Two, How much can class weighting improve balanced performance on a genre distribution that is strongly skewed? Three, Which genres are actually difficult to identify? More than the overall accuracy, macro precision, recall, and F1-score are emphasized so that performance on the minor Dance and Heavy Metal classes is not hidden by the major Rock and Pop classes.

The most obvious finding is that the contextual pretrained transformers significantly outperform the recurrent baselines in macro F1. Weighted RoBERTa produces the highest single-model accuracy, while averaging the probabilities of unweighted BERT, weighted BERT, and weighted RoBERTa gives the best overall result. However, the remaining class-specific errors show that data

imbalance and longer document handling are still the main obstacles.

2 Related Work

Lyric depended genre classification work has traditionally depended on lexical count and manually designed features. Mayer et al. (2008) combined bag-of-words evidence with rhyme, part-of-speech, and statistical features, showcasing genre cues beyond word frequencies that lyrics contain. Fell and Sporleder (2014) found topical n-grams and stylistic features to be complementary by examining genre, quality. These are the findings that encourage our study that compares sparse lexical features and learned representations.

Word2Vec learns continuous word vectors from local prediction (Mikolov et al., 2013). By averaging these vectors we create a compact document representation, although order is removed and it can blur local signals. Recurrent networks are introduced to retain sequence information. The sequence information enables recurrent models to reach great performance on several NLP tasks but for this specific task with the given dataset, the recurrent models suffer from overlapping sequences and severe class imbalance.

Transformers replace recurrence with self-attention (Vaswani et al., 2017). BERT pretrains bidirectional contextual representations and further adapts them to classification tasks by fine-tuning the encoder with a task-specific output layer (Devlin et al., 2019). RoBERTa builds on BERT’s pre-training procedure but differentiates itself through different choices such as removing next-sentence prediction step and using more data and in larger batches (Liu et al., 2019). This project evaluates both base-sized models on the same six-class dataset and further tests with class weights to reduce error caused by severe class imbalance. Finally, ensemble technique is adopted to utilize all of the different models for the best outcome.

3 Methodology

3.1 Task and Dataset

The dataset was collected from the public Auto NLP song-lyrics collection on Hugging Face (Simon, n.d.). Two raw CSV files were combined, producing 53,882 English lyrics with labels. Each row had the lyric text and one of six labels. There were no missing text or label values. The initial

Genre	Instances	Share (%)
Rock	21,043	39.25
Pop	12,325	22.99
Hip Hop	9,792	18.27
Indie	5,668	10.57
Heavy Metal	3,032	5.66
Dance	1,749	3.26
Total	53,609	100.00

Table 1: Class distribution after cleaning.

audit found 201 fully duplicated rows. For deduplication, pandas drop_duplicates function was used using lyric text only. 273 repeated lyric texts were removed in total. Among them 72 appeared with conflicting genre labels. The resulting corpus had 53,609 unique lyric texts.

Table 1 shows severe class imbalance. Rock is more than twelve times as frequent as Dance. Therefore, the need for computing balanced class weights was justified. The computed weights were 5.1085 for Dance, 2.9468 for Heavy Metal, 0.9125 for Hip Hop, 1.5764 for Indie, 0.7249 for Pop, and 0.4246 for Rock. The weights were used by models that supported them.

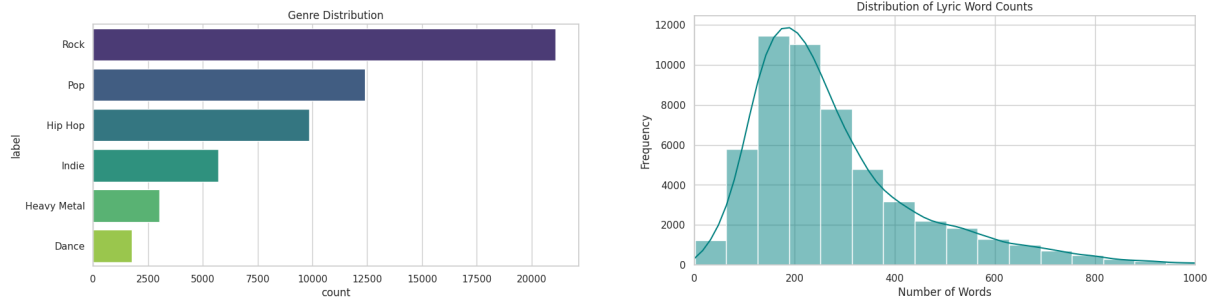
3.2 Exploratory Data Analysis

Figure 1 presents the two quantitative EDA plots showcasing imbalanced genre distribution and lyric word-count distribution. Lyrics contained an average of 282 words, and the 90th percentile was 528 words. The word-count histogram shows a good chunk of texts extending beyond the input limit of 256-tokens used by sequence and transformer models.

The comparative word clouds in Figure 2 check genre-specific lexical characteristics. Common structural lyric markers and conversational fillers were excluded for this visualization. Heavy Metal and Pop still share many high-frequency words between them, including terms related to love, knowing, and time. This overlap of vocabulary among genres helps to understand why vocabulary alone is not enough and why models tends to confuse neighboring genres.

3.3 Preprocessing and Data Splits

Two separate preprocessing strategies were adopted in the project. The heavy-cleaning pipeline lower-cased text, removed bracketed section markers, normalized newlines, removed punctuation, removed English stopwords, and lemmatized the tokens.



This set of data was used for the TF-IDF and Word2Vec experiments. The minimal pipeline did not remove case, punctuation, stopwords, and structural markers but removed excess whitespace. This set was used for the recurrent and transformer experiments because those models can use syntax and contextual data. NLTK provided stopwords and lemmatization resources were used for preprocessing steps (Bird et al., 2009).

A two-stage stratified split with random seed 42 was used to separate 80% of the data for training and 10% each for validation and testing. After the split, we got 42,887 training, 5,361 validation, and 5,361 test rows. Feature extraction and vocabulary fitting used only the training split. Validation data was used to guide models and for checkpoint selection, while the testing split was untouched until final evaluation.

3.4 Text Representations

TF-IDF represent a document using term frequency inversed by corpus-level document frequency, following the idea that rare terms that appear in a given class are more specific to the

class(Sp"arck Jones, 1972). We limited the vectorizer to 5,000 features which is still a sparse representation. Word2Vec was trained on training lyrics with 100 dimensions, with context window of 5, and minimum token frequency of 2. This yielded a dense 100-dimensional representation but lost word order in the process.

The recurrent models used a different approach. A tokenizer used the 20,000 most frequent terms, and mapped the unseen terms to an out-of-vocabulary token ("`<ooov>`"), and post-padded or truncated every sequence to 256 positions. BERT and RoBERTa used their native subword tokenizers with the same maximum 256-token and dynamic batch padding.

3.5 Models

The classic ml experiments used logistic regression, Naive Bayes, and random forest implementations from scikit-learn (Pedregosa et al., 2011). Logistic regression and random forest were trained and tested with both TF-IDF and Word2Vec representations. Multinomial Naive Bayes was applied to TF-IDF features, while Gaussian Naive Bayes was

applied to dense Word2Vec vectors.

Six recurrent classifiers were implemented, namely SimpleRNN, GRU, LSTM, bidirectional SimpleRNN, bidirectional GRU, and bidirectional LSTM. Each model was configured with a trainable embedding layer, one recurrent layer, dropout, and a linear six-class output layer. Training used Adam as optimizer, batch size of 128, class-weighted cross-entropy, maximum 20 epochs, and early stopping with patience 3 on validation loss.

Transformer experiments utilized the HuggingFace Transformers library for collecting pretrained models (Wolf et al., 2020). BERT-base-uncased and RoBERTa-base were fine-tuned with learning rate of 2×10^{-5} , batch size of 64, weight decay of 0.01, half-precision (fp16) training, and validation-based checkpoint selection. The baseline BERT run used ordinary cross-entropy loss for at most 10 epochs. Weighted BERT trained for max 20 epochs with class-weighted cross-entropy loss. RoBERTa was trained for max 20 epochs with the weighted loss. Finally, the ensemble technique averaged the softmax probabilities of all three trained models, unweighted BERT, weighted BERT, and weighted RoBERTa, and selected the class with the largest average probability.

3.6 Hyperparameter Selection

Each classical model and each recurrent architecture was trained with three manually specified configurations. Logistic regression used different $C \in \{0.1, 1, 10\}$ values, while Naive Bayes used different smoothing, and random forest used different tree count and maximum depth. The recurrent search used three different configuration of embedding/hidden/dropout settings of (128, 128, 0.3), (128, 256, 0.4), and (256, 128, 0.5). Classical and Recurrent configurations were compared using validation macro F1, with validation loss overseeing early stopping for recurrent models. Full recorded runs are reported in Appendix A.

The transformer section compared base and weighted BERT training and a weighted RoBERTa training under fixed optimization settings for all. This smaller set of configuration was used due to computational and time restrictions. There was an attempt to include DeBERTa in this scenario but the training loss was exploding under given settings, hence dropped later.

4 Experimental Setup

All the best selected models were tested on the same 5,361 test partition. Accuracy measures the fraction of correct predictions. Because the test set is imbalanced, macro precision, macro recall, and macro F1 are reported by first calculating each metric independently for every genre and then averaging the six values. Macro F1 was selected as the primary selection metric because each label receives equal weight.

The implementation used pandas and NumPy for data preparation. Scikit-learn was used for splitting, representations, classical models, and metrics. Gensim was used for Word2Vec (Rehurek and Sojka, 2010), PyTorch was used for recurrent networks for enabling GPU acceleration of latest blackwell architecture, and Hugging Face Transformers used to get the pretrained language models. Matplotlib and Seaborn generated the graphs and visualizations (Hunter, 2007; Waskom, 2021). The training ran on an NVIDIA GeForce RTX 5090 with 32GB VRAM. Random seeds set to 42 where applicable.

5 Results and Discussion

5.1 Overall Comparison

Table 2 combines the test results. The three-model ensemble achieved the highest accuracy (0.66) and macro F1 (0.56). Weighted RoBERTa model achieved 0.61 accuracy and 0.53 macro F1. The BERT test output reports 0.64 accuracy and 0.52 macro F1. Weighted BERT model achieved 0.63 accuracy and 0.54 macro F1, beating all the other independent models.

The classical results show that accuracy does not reveal the whole picture. Multinomial Naive Bayes reaches 0.56 accuracy, but its macro F1 was only 0.31. It recalled 91% of Rock examples while recalling none of Indie and about 1% of Dance and Heavy Metal. Logistic regression shows a reduction in overall accuracy to 0.45 but obtained the best classical macro F1 because the predictions were distributed more evenly. Random forest, on the other hand, offered 0.55 accuracy and 0.40 macro F1.

The recurrent models did not show proper improvement over the classical baselines. Bidirectional GRU was the best recurrent architecture at 0.37 macro F1, followed by GRU at 0.33. Both LSTM versions reached 0.31 macro f1. SimpleRNN and bidirectional SimpleRNN were the

Family	Model and selected setting	Accuracy	Macro P	Macro R	Macro F1
Classical	Logistic regression, TF-IDF	0.45	0.42	0.47	0.41
Classical	Multinomial NB, TF-IDF	0.56	0.47	0.33	0.31
Classical	Random forest, TF-IDF	0.55	0.52	0.40	0.40
Recurrent	SimpleRNN	0.23	0.28	0.31	0.18
Recurrent	Bidirectional SimpleRNN	0.23	0.24	0.30	0.18
Recurrent	GRU	0.35	0.35	0.40	0.33
Recurrent	Bidirectional GRU	0.41	0.39	0.44	0.37
Recurrent	LSTM	0.34	0.35	0.38	0.31
Recurrent	Bidirectional LSTM	0.37	0.33	0.35	0.31
Transformer	BERT unweighted*	0.64	0.59	0.51	0.52
Transformer	RoBERTa-base, weighted	0.61	0.58	0.53	0.53
Ensemble	BERT $\times 2$ + RoBERTa soft voting	0.66	0.64	0.53	0.56

Table 2: Test-set comparison of all the models.

weakest of them all at 0.18 macro F1. The recurrent confusion matrices in Appendix B reveal unstable class preferences. One likely contributor is the combination of post-padding and a final-time-step readout. For lyrics shorter than 256 tokens, that readout can correspond primarily to padding rather than the last lexical token.

5.2 Effect of Class Weighting and Ensembling

Weighted BERT model converged later than the unweighted BERT model. The impacts of the class weights is significant if we look at individual class performance. The weighted model performs much better on the minority classes like Dance and Heavy Metal compared to the unweighted model. But it comes at a accuracy cost, as unweighted model achieves higher accuracy by guessing the popular classes more.

Weighted RoBERTa further increased the performance for the minority classes but took a bigger hit in the overall accuracy. This result perfectly validates our call for ensemble technique as we have two weighted models that protect the minority classes and one unweighted model that performs according to the dataset distribution. The combined result produced the highest performance among all independent models and our competitor Auto NLP model that achieved 67% accuracy compared to 66% of ours but sacrificed F1 score (53% vs. 56%).

5.3 Error Analysis

Figure 3 shows that Hip Hop is the most accurately predicted class. The ensemble correctly classified 803 of 979 Hip Hop examples and achieved class F1 of 0.86. Rock as the most popular class did better with 1,644 correct ensemble predictions out

of 2,104. In contrast, Indie lyrics were frequently classified as Rock (280 times). Heavy Metal is also often confused with Rock (162 case), while 359 Pop lyric were assigned to Rock.

These patterns aligns with the hypothesis of the EDA. The imbalance class distributions presents many more example of broad Rock and Pop languages than of Dance or Heavy Metal. The word clouds also show that broad emotional terms such as "love", are shared between otherwise different genres. Moreover, a lot of lyrics are longer than the 256-token limit resulting in being truncated. Truncation omits late verses or repeated choruses that carry genre specific cues. The remaining errors points to ambiguity in the data itself.

5.4 Answers to the Research Questions

For the first research question, pretrained contextual transformer encoders (e.g. BERT) are the strongest representation and architecture combination. The independent models and later the ensemble technique out performed all the classic ML models and recurrent models. Classical TF-IDF models remained competitive baselines. Logistic regression outperformed all six selected recurrent systems on macro F1. Word2Vec validation scores do not surpass the selected TF-IDF variants.

For the second research question, the unweighted BERT base model achieves 0.52 F1 and 0.64 accuracy compared to the weighted BERT base models 0.54 F1 and 0.63 accuracy. The class weights improved the F1 score by a good margin while loosing some accuracy in the process. Overall, the weighted model is more preferable as it does not completely ignore the minority classes like the unweighted model. Computed class weights remain a strong candidate to reduce errors regarding

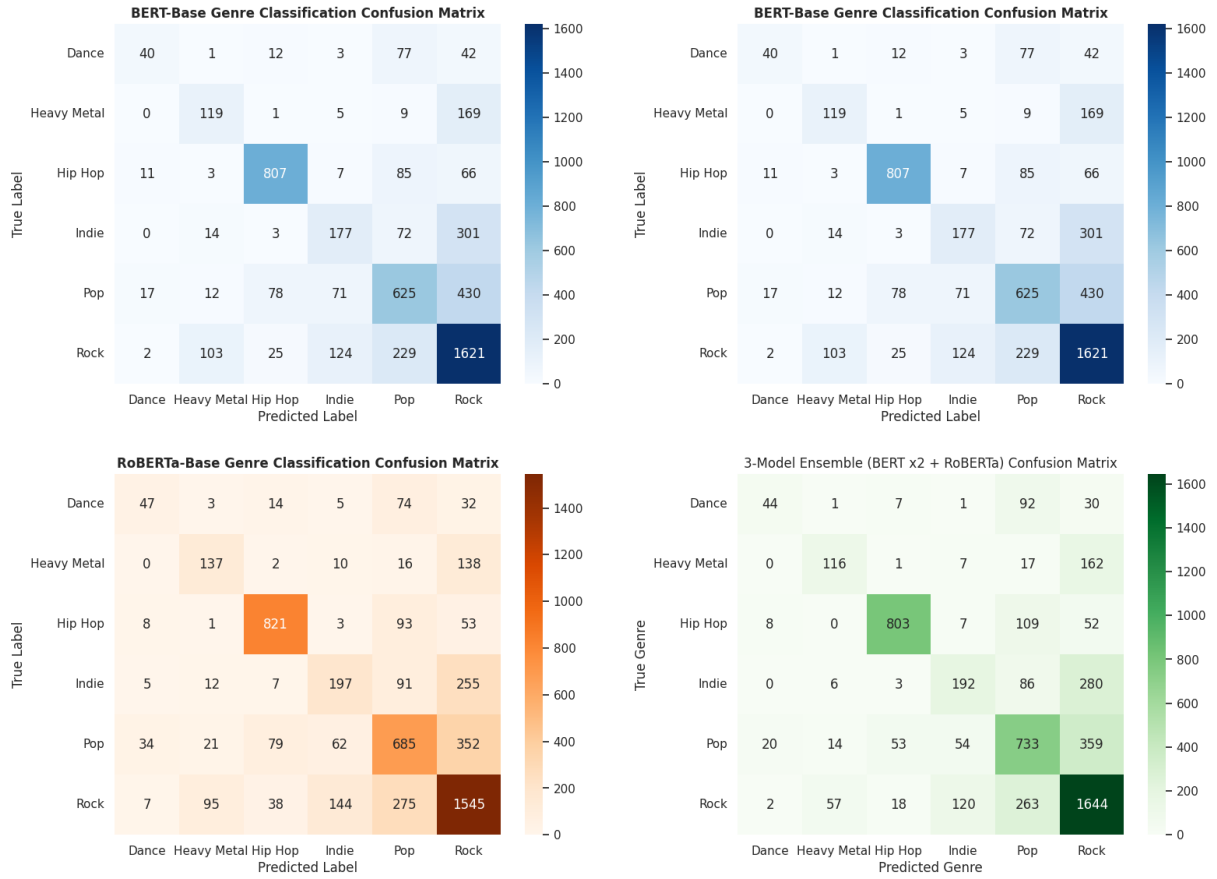


Figure 3: All four transformer-stage matrices.

dataset class imbalance.

For the third and final research question, Hip Hop has been proved to be the easiest class to identify correctly and the minority genres remain the hardest to predict. Hip Hop has a distinctive vocabulary compared to other genres resulting in high recall across model families, on the other hand, Dance, Heavy Metal, and Indie have very few training examples and comparatively greater lexical overlap with the popular Rock and Pop genres. The confusion matrices in Figure 3 show that class weights can increase minority predictions and reduce this gap but cannot eliminate the overlap problem.

6 Limitations and Future Work

The results are limited by several factors. First of all, despite the fact 72 repeated lines appeared with contradictory and conflicting labels, the dataset describes a genre very ambiguously and clearly showing annotation discrepancy. Secondly, class weights do not completely replace the need for a more balanced data or thorough resampling due to the dataset’s extreme imbalance. Third, the re-

current approach can determine based on the final position of post padded sequences but averaging Word2Vec vectors that ignores order. A more balance recurrent comparison possible with packed sequences or mask-aware pooling.

Fourth, many lyrics were truncated to 256 token constraint and with subword expansion. Most of the instances were heavily truncated as we previously observed 90th-percentile length of 528 words. More information would be retained if we could use longer sequence length like 512 tokens. Fifth, rather than evaluating three configuration for each architecture, the transformers tuning was fixed to predefined optimization settings.

Padding aware recurrent summaries might be contrasted with longer input and hierarchical pooling. Testing calibrated or validation weighted ensembles can be helpful. Stopword removal, class weighting, structural lyrics tags and each ensemble component may all be measured independently in ablation studies. Lastly, repeating stratified runs would help to determine if the benefits are indeed significant or just a result of variance from a single random split.

7 Conclusion

This project made comparison between conventional machine learning, recurrent networks, pre-trained transformers and a three-model soft voting ensemble to classify the genres of six class based music lyrics. Based on the project research the ensemble produced the best test macro F1 of 0.56 with an accuracy of 0.66. Among the basic traditional models logistic regression produced better results and the bidirectional GRU was the best recurrent model.

The gap between macro F1 and accuracy is highly visible specially for Naive Bayes and showing that when the dataset is very unbalanced the accuracy is insufficient on its own. At the end, the research study highlights that contextual pre-training performed better than the basic recurrent models and sparse feature out performed averaged representations in captureing genre related patterns. There are still so many gaps that exists for development. Increasing the performance and improving the reliability of the project findings might be achieved by making adjustment to the BERT evaluation, employing longer and broader context, having clearer labels and doing more thorough tuning.

References

- Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics.
- Michael Fell and Caroline Sporleder. 2014. [Lyrics-based analysis and classification of music](#). In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 620–631, Dublin, Ireland. Dublin City University and Association for Computational Linguistics.
- John D. Hunter. 2007. [Matplotlib: A 2d graphics environment](#). *Computing in Science and Engineering*, 9(3):90–95.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. [RoBERTa: A robustly optimized BERT pretraining approach](#). *arXiv preprint arXiv:1907.11692*.
- Rudolf Mayer, Robert Neumayer, and Andreas Rauber. 2008. [Rhyme and style features for musical genre classification by song lyrics](#). In *Proceedings of the 9th International Conference on Music Information Retrieval*, pages 337–342.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. [Efficient estimation of word representations in vector space](#). *arXiv preprint arXiv:1301.3781*.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. [Scikit-learn: Machine learning in python](#). *Journal of Machine Learning Research*, 12(85):2825–2830.
- Radim Rehurek and Petr Sojka. 2010. [Software framework for topic modelling with large corpora](#). In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50.
- Julien Simon. n.d. [AutoNLP dataset for project: Song lyrics demo](#). Hugging Face Datasets. Accessed 2026-08-28.
- Karen Spärck Jones. 1972. [A statistical interpretation of term specificity and its application in retrieval](#). *Journal of Documentation*, 28(1):11–21.
- Alexandros Tsaptsinos. 2017. [Lyrics-based music genre classification using a hierarchical attention network](#). *arXiv preprint arXiv:1707.04678*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, volume 30.
- Michael L. Waskom. 2021. [seaborn: Statistical data visualization](#). *Journal of Open Source Software*, 6(60):3021.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. [Transformers: State-of-the-art natural language processing](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics.

A Complete Hyperparameter Records

Model	Representation	Configuration	Val. Acc.	Macro F1	Weighted F1
Logistic regression	TF-IDF	$C = 0.1$, balanced	.4122	.3840	.4363
Logistic regression	TF-IDF	$C = 1.0$, balanced	.4397	.3992	.4638
Logistic regression	TF-IDF	$C = 10.0$, balanced	.4441	.3937	.4666
Logistic regression	Word2Vec	$C = 0.1$, balanced	.3826	.3520	.4059
Logistic regression	Word2Vec	$C = 1.0$, balanced	.3816	.3519	.4044
Logistic regression	Word2Vec	$C = 10.0$, balanced	.3815	.3517	.4042
Multinomial NB	TF-IDF	$\alpha = 0.1$	.5590	.3067	.4861
Multinomial NB	TF-IDF	$\alpha = 0.5$	.5561	.3018	.4811
Multinomial NB	TF-IDF	$\alpha = 1.0$	.5547	.2977	.4766
Gaussian NB	Word2Vec	smoothing 10^{-9}	.3796	.3032	.3805
Gaussian NB	Word2Vec	smoothing 10^{-7}	.3796	.3032	.3805
Gaussian NB	Word2Vec	smoothing 10^{-5}	.3796	.3032	.3805
Random forest	TF-IDF	100 trees, depth 15	.4145	.3678	.4176
Random forest	TF-IDF	200 trees, depth 30	.5506	.3964	.5290
Random forest	TF-IDF	300 trees, unlimited depth	.5741	.3356	.4954
Random forest	Word2Vec	100 trees, depth 15	.5411	.3619	.5034
Random forest	Word2Vec	200 trees, depth 30	.5555	.3309	.4864
Random forest	Word2Vec	300 trees, unlimited depth	.5536	.3219	.4820

Table 3: All the different configurations for classical models

Architecture	Embedding	Hidden	Dropout	Final logged validation accuracy
SimpleRNN	128	128	.3	.2328
SimpleRNN	128	256	.4	.2166
SimpleRNN	256	128	.5	.4557 (selected)
Bidirectional SimpleRNN	128	128	.3	.4467 (selected)
Bidirectional SimpleRNN	128	256	.4	.2279
Bidirectional SimpleRNN	256	128	.5	.2729
GRU	128	128	.3	.4068 (selected)
GRU	128	256	.4	.3796
GRU	256	128	.5	.3193
Bidirectional GRU	128	128	.3	.3593
Bidirectional GRU	128	256	.4	.4492 (selected)
Bidirectional GRU	256	128	.5	.3740
LSTM	128	128	.3	.3693 (selected)
LSTM	128	256	.4	.3300
LSTM	256	128	.5	.3663
Bidirectional LSTM	128	128	.3	.3180 (selected)
Bidirectional LSTM	128	256	.4	.3134
Bidirectional LSTM	256	128	.5	.3179

Table 4: Recurrent search configurations.

Run	Epochs done	Best epoch	Checkpoint acc.	Best val. weighted F1	Loss
BERT-base	7	4	.6234	.6092	Unweighted
BERT-base, independent	15	12	.6049	.6036	Class weighted
RoBERTa-base	10	7	.6240	.6182	Class weighted

Table 5: Transformer validation histories.

B All Notebook Confusion-Matrix Grids

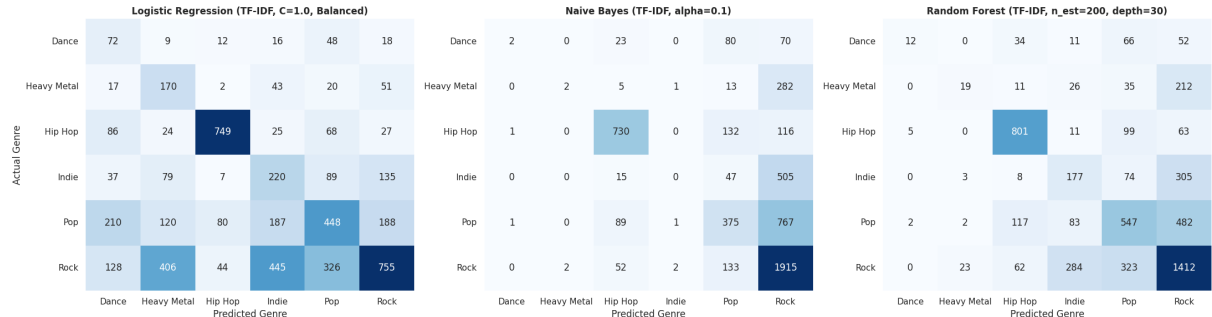


Figure 4: Complete classical-model confusion-matrix grid.



Figure 5: Complete recurrent-model confusion-matrix grid.

C Full Classification Reports

Model	Class	Precision	Recall	F1	Support
Logistic regression	Dance	.13	.41	.20	175
	Heavy Metal	.21	.56	.31	303
	Hip Hop	.84	.77	.80	979
	Indie	.24	.39	.29	567
	Pop	.45	.36	.40	1,233
	Rock	.64	.36	.46	2,104
Multinomial NB	Dance	.50	.01	.02	175
	Heavy Metal	.50	.01	.01	303
	Hip Hop	.80	.75	.77	979
	Indie	.00	.00	.00	567
	Pop	.48	.30	.37	1,233
	Rock	.52	.91	.67	2,104
Random forest	Dance	.63	.07	.12	175
	Heavy Metal	.40	.06	.11	303
	Hip Hop	.78	.82	.80	979
	Indie	.30	.31	.31	567
	Pop	.48	.44	.46	1,233
	Rock	.56	.67	.61	2,104

Table 6: Per-class reports for the selected classical models.

Model	Class	Precision	Recall	F1	Support
SimpleRNN	Dance	.07	.33	.11	175
	Heavy Metal	.12	.03	.05	303
	Hip Hop	.52	.71	.60	979
	Indie	.15	.77	.25	567
	Pop	.34	.04	.07	1,233
	Rock	.47	.00	.01	2,104
Bidirectional SimpleRNN	Dance	.07	.28	.12	175
	Heavy Metal	.06	.03	.04	303
	Hip Hop	.48	.65	.55	979
	Indie	.15	.77	.25	567
	Pop	.39	.09	.15	1,233
	Rock	.29	.00	.00	2,104
GRU	Dance	.10	.35	.15	175
	Heavy Metal	.12	.53	.20	303
	Hip Hop	.77	.79	.78	979
	Indie	.21	.25	.23	567
	Pop	.36	.34	.35	1,233
	Rock	.56	.16	.25	2,104
Bidirectional GRU	Dance	.10	.45	.17	175
	Heavy Metal	.16	.57	.26	303
	Hip Hop	.80	.76	.78	979
	Indie	.26	.22	.24	567
	Pop	.42	.30	.35	1,233
	Rock	.57	.34	.43	2,104

Table 7: Per-class reports for SimpleRNN and GRU variants.

Model	Class	Precision	Recall	F1	Support
LSTM	Dance	.09	.25	.13	175
	Heavy Metal	.12	.29	.17	303
	Hip Hop	.76	.73	.74	979
	Indie	.19	.47	.27	567
	Pop	.36	.39	.37	1,233
	Rock	.56	.12	.19	2,104
Bidirectional LSTM	Dance	.09	.27	.14	175
	Heavy Metal	.08	.09	.08	303
	Hip Hop	.69	.69	.69	979
	Indie	.18	.51	.27	567
	Pop	.39	.21	.27	1,233
	Rock	.53	.31	.39	2,104

Table 8: Per-class reports for LSTM variants.